

SIMATS ENGINEERING
Co3-ASSESSMENT TOOL 2
Case study

COURSE NAME: Computer Networks

COURSE CODE: CSA07

COURSE OUTCOME COVERED

CO3: Analyze the performance of core network protocols such as TCP, UDP, HTTP, and DNS under varying conditions

Assessment Tool Weightage: 40%

Dr.V.Parthipan

QUESTIONS:4

Total Marks:40

Code of Conduct:

I _ M.Greeshma(192524195)___ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: M.Greeshma

1. A multinational IT company uses a cloud-based video conferencing platform for daily client meetings. During peak business hours, employees experience frequent voice distortion, frozen video frames, and delayed communication. Network monitoring reports indicate an 8% packet loss, 180 ms jitter, and fluctuating bandwidth utilization. The IT administrator suspects that the issue is related to the transport protocol or application-level buffering strategy.

Questions

- 1. Analyze whether the problem is primarily caused by the transport protocol (TCP/UDP) or the application layer.**
- 2. Justify your answer based on the communication requirements of real-time multimedia applications.**
- 3. Recommend suitable protocol-level or application-level improvements to enhance conferencing quality.**

Sol....

1. Root Cause: Transport Protocol vs Application Layer

- **Transport Protocol (TCP/UDP):**
 - Real-time conferencing platforms almost always use UDP for audio/video streams because it avoids retransmission delays.
 - The reported 8% packet loss and 180 ms jitter are severe for UDP-based real-time traffic. Unlike TCP, UDP does not retransmit lost packets, so packet loss directly causes voice distortion and frozen frames.
 - TCP would worsen latency due to retransmissions, so it's unlikely the platform is relying on TCP for the media stream.
- **Application Layer:**
 - The application's buffering and error concealment strategies determine how well it masks packet loss and jitter.

- With high jitter and packet loss, if the application's jitter buffer is too small, frames arrive late and get dropped. If it's too large, latency increases, causing delayed communication.
- Therefore, while the transport protocol (UDP) exposes the system to packet loss/jitter, the application layer buffering strategy determines how severe the user experience degradation is.

Conclusion: The primary issue is application-layer buffering and error handling, aggravated by poor network conditions (packet loss/jitter). UDP is the correct choice for real-time conferencing, but the app's adaptation mechanisms are insufficient.

2. Communication Requirements of Real-Time Multimedia

- Low Latency: Real-time audio/video requires end-to-end delay <150 ms for natural conversation. TCP retransmissions would break this requirement.
- Tolerance to Loss: Multimedia can tolerate some packet loss if concealment techniques (e.g., forward error correction, interpolation) are used.
- Consistency: Jitter must be smoothed by adaptive jitter buffers; otherwise, playback stalls.

Thus, UDP is the right transport protocol, but the application must compensate for loss and jitter with smart buffering and recovery.

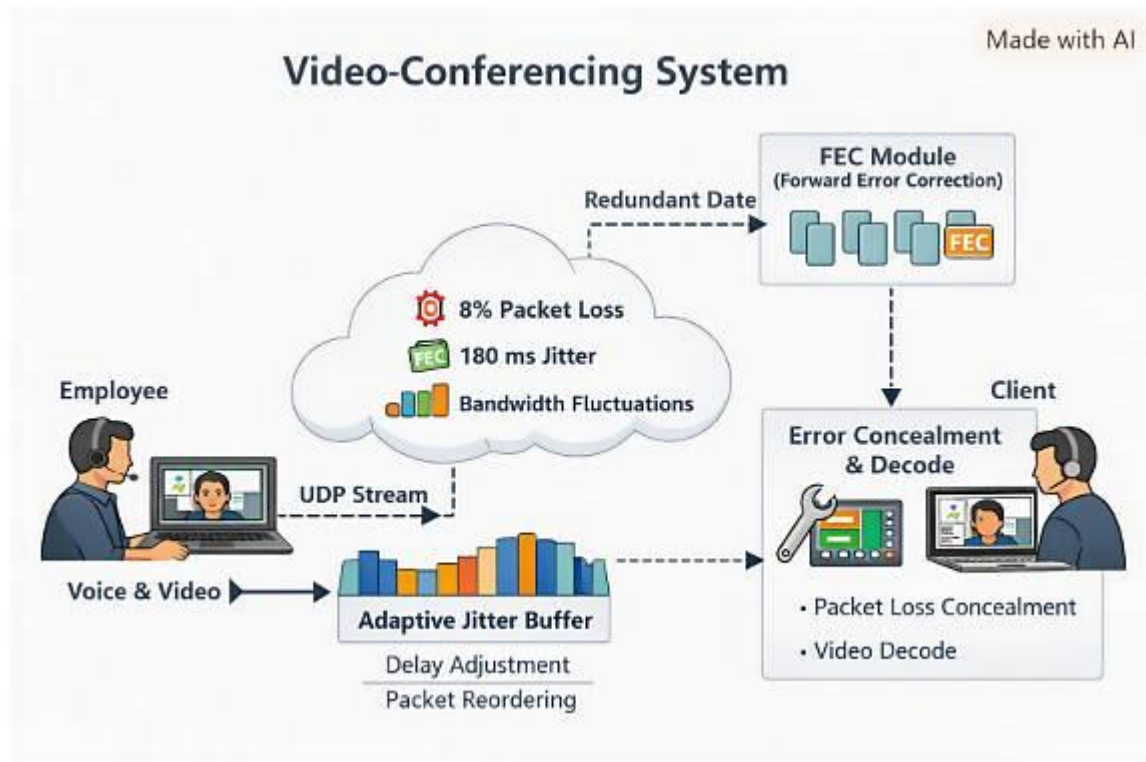
3. Recommendations for Improvement

Protocol-Level:

- Implement Forward Error Correction (FEC) to recover lost packets without retransmission.
- Use Selective Retransmission for key frames (hybrid approach).
- Enable QoS (Quality of Service) at the network level to prioritize conferencing traffic.

Application-Level:

- Deploy adaptive jitter buffers that dynamically resize based on network conditions.
- Use packet loss concealment (PLC) techniques for audio (e.g., waveform substitution, interpolation).
- Apply scalable video coding (SVC) to gracefully degrade video quality under bandwidth fluctuations.
- Introduce bandwidth estimation algorithms (e.g., Google Congestion Control in WebRTC) to adapt bitrate in real time.



2. A multinational software company hosts its web services across multiple continents. Employees frequently report long delays before the application homepage loads. Network analysis shows that the average DNS lookup time is approximately 800 ms, significantly affecting user experience. The organization plans to redesign its DNS infrastructure while ensuring continuous service availability during server failures.

Questions

1. Analyze the reasons behind the high DNS resolution time.
2. Propose a suitable DNS architecture using techniques such as Anycast DNS, DNS pre-fetching, and TTL optimization to reduce the lookup time below 50 ms.
3. Evaluate the advantages and limitations of your proposed architecture in terms of performance, scalability, and availability.

Sol...

1.Causes of High DNS Resolution Time (≈800 ms)

Factor	Description	Impact
Geographical distance	DNS queries may be routed to authoritative servers located on other continents.	High latency due to long round-trip times.
Lack of caching	If TTL values are too short, clients and recursive resolvers must repeatedly query authoritative servers.	Increased lookup frequency and delay.
Unoptimized resolver path	Recursive resolvers may not use the nearest or fastest upstream DNS server.	Adds multiple hops and processing delays.
No Anycast or CDN integration	DNS servers may be centralized instead of globally distributed.	Poor performance for remote users.
Network congestion or packet loss	Causes retransmissions and timeouts during DNS queries.	Further increases lookup time.

2. Proposed DNS Architecture

Goal: Reduce DNS lookup time below 50 ms while maintaining high availability.

Architecture Components

Technique	Implementation	Benefit
Anycast DNS	Deploy DNS servers with identical IP addresses across global PoPs (Points of Presence). Routing automatically directs users to the nearest server.	Minimizes latency by geographic proximity; improves fault tolerance.
DNS Prefetching	Configure browsers and applications to pre-resolve domain names likely to be accessed soon (e.g., homepage assets).	Reduces perceived delay for subsequent requests.
TTL Optimization	Set TTL values strategically: longer for static records (e.g., homepage), shorter for dynamic ones. Example: 1 hour for static, 5 minutes for dynamic.	Balances caching efficiency and flexibility during updates.
Load Balancing & Failover	Use multiple authoritative servers behind Anycast with health checks.	Ensures continuous service during server failures.
DNSSEC with minimal overhead	Maintain security without excessive computational delay.	Protects integrity while keeping resolution fast.

3. Evaluation of the Proposed Architecture

Criterion	Advantages	Limitations
Performance	Anycast reduces latency to <50 ms globally; caching and prefetching minimize repeated lookups.	Requires global infrastructure and routing optimization.
Scalability	Easily scales by adding new PoPs; Anycast routing automatically balances load.	Complex to manage DNS record synchronization across nodes.
Availability	Redundant Anycast nodes and failover ensure near-zero downtime.	Misconfigured routing policies could cause uneven traffic distribution.

3.A cloud service provider delivers applications to millions of users worldwide.

During promotional events, users experience intermittent delays in accessing cloud-hosted services. Investigation reveals that DNS queries are taking nearly

800 ms because all requests are directed to a single primary DNS server. The company wants a highly available DNS solution capable of handling global traffic efficiently.

Questions

4. Examine the impact of centralized DNS architecture on application performance.

5. Design a distributed DNS solution that minimizes lookup latency while maintaining high availability.

6. Analyze how TTL values, Anycast routing, and DNS caching influence system performance and fault tolerance.

Sol..

4. Impact of Centralized DNS Architecture on Application Performance

Issue	Description	Effect
Single Point of Failure	All DNS queries depend on one primary server. If it's overloaded or fails, resolution halts.	Causes intermittent delays or complete service outage.
High Latency for Global Users	Users far from the server experience long round-trip times.	DNS lookup times can exceed 800 ms, slowing page loads.
Limited Scalability	One server cannot efficiently handle millions of concurrent queries.	Performance degrades during traffic spikes (e.g., promotions).
Poor Fault Tolerance	No redundancy or failover mechanism.	Reduced reliability and availability.

Summary: Centralization creates bottlenecks and latency, directly degrading user experience and application responsiveness.

5. Distributed DNS Solution Design

Objective: Achieve global low-latency resolution (< 50 ms) and high availability.

Key Components

Technique	Implementation	Benefit
Anycast DNS	Deploy identical DNS servers across continents sharing one IP address. Routing directs queries to the nearest node.	Minimizes latency and balances global load.
Geo-Distributed Authoritative Servers	Regional clusters synchronize records via secure replication.	Enhances scalability and fault tolerance.
DNS Caching	Recursive resolvers and CDN edge nodes cache responses based on TTL.	Reduces repeated lookups and server load.
Load Balancing & Health Checks	Monitors node health; reroutes traffic automatically during failures.	Ensures continuous service availability.
TTL Optimization	Longer TTLs for static records, shorter for dynamic ones.	Balances performance and flexibility during updates.

6. Influence of TTL, Anycast Routing, and DNS Caching

Factor	Role	Performance Impact	Fault-Tolerance Impact
TTL (Time-to-Live)	Determines how long DNS records are cached.	Longer TTLs reduce lookup latency; shorter TTLs allow faster updates.	Long TTLs maintain service continuity during transient failures.
Anycast Routing	Routes queries to the nearest available DNS node.	Reduces latency by geographic proximity; improves load distribution.	Automatic rerouting during node failure enhances resilience.
DNS Caching	Stores resolved records locally or at edge servers.	Eliminates repeated queries; improves response time.	Cached data ensures continuity even if upstream servers fail temporarily.

4.A financial institution operates an electronic stock trading platform where thousands of buy and sell orders are submitted every second. During market opening hours, the average order acknowledgment latency increases from 1 ms to 40 ms, resulting in delayed trade confirmations and customer dissatisfaction. Network engineers observe increased TCP retransmissions, longer connection establishment times, and excessive buffering.

Questions

1. Analyze three possible TCP-related factors responsible for the latency increase, considering flow control, Nagle’s algorithm, and SYN queue limitations.
2. Explain how each factor affects transaction processing in high-frequency systems
3. Recommend appropriate TCP configuration changes to reduce latency and improve transaction reliability.

Sol..

1. TCP-Related Factors Behind Latency Increase

1. Flow Control (TCP Window Size & Congestion Control)

- When traffic spikes, TCP's sliding window shrinks due to congestion signals.
- This throttles throughput, forcing senders to wait for acknowledgments before transmitting more data.

2. Nagle's Algorithm

- Designed to reduce small packet overhead by batching tiny segments until an ACK is received.
- In high-frequency trading, this introduces micro-delays that accumulate into noticeable latency.

3. SYN Queue Limitations

- At market open, thousands of new connections flood the server.
- The SYN backlog queue may overflow, causing dropped or delayed connection establishment.
- This increases handshake time and delays order acknowledgments.

2. Impact on High-Frequency Transaction Processing

Factor	Effect on Trading System
Flow Control	Slows down order transmission under congestion, delaying confirmations.
Nagle's Algorithm	Aggregates small trade packets, introducing milliseconds of delay per order.
SYN Queue Limits	Causes connection setup delays or failures, preventing timely order submission.

3. Recommended TCP Configuration Changes

1. Disable Nagle's Algorithm

- Use the TCP_NODELAY socket option.
- Ensures immediate transmission of small trade packets without batching.

2. Optimize Flow Control

- Increase TCP window size and tune congestion control algorithms (e.g., BBR or CUBIC).
- Use low-latency queuing and prioritize trading traffic at the network layer.

3. Expand SYN Queue Capacity

- Increase tcp_max_syn_backlog and enable SYN cookies to handle bursts of new connections.
- Consider persistent connections or connection pooling to reduce handshake overhead.

4. Additional Enhancements

- Deploy kernel bypass technologies (e.g., DPDK, RDMA) for ultra-low latency.
- Use dedicated trading network segments with QoS to isolate critical traffic.